

Лабораторная работа №4: Объектно-ориентированное программирование (ООП)

Часть 1: Основы классов и объектов

Класс – это шаблон для создания объектов. Объект – экземпляр класса.

Простейший класс

```
class Dog:
```

```
    """Класс, представляющий собаку"""
```

```
    def __init__(self, name, age):
```

```
        """Конструктор – вызывается при создании объекта"""
```

```
        self.name = name # атрибут экземпляра
```

```
        self.age = age
```

```
    def bark(self):
```

```
        """Метод экземпляра"""
```

```
        print(f"{self.name} говорит: Гав-гав!")
```

```
    def get_human_age(self):
```

```
        """Возвращает возраст собаки в пересчёте на человеческий"""
```

```
        return self.age * 7
```

Создание объектов

```
my_dog = Dog("Рекс", 3)
```

```
your_dog = Dog("Бобик", 5)
```

```
print(my_dog.name)          # Рекс
```

```
my_dog.bark()               # Рекс говорит: Гав-гав!
```

```
print(f"Возраст по-человечески: {my_dog.get_human_age()}") # 21
```

Задание 1.1: Создайте класс Book с атрибутами: title, author, year, is_available (по умолчанию True). Добавьте методы:

- `borrow()` – меняет статус на `False`, если книга доступна, иначе выводит сообщение.
- `return_book()` – меняет статус на `True`.
- `get_info()` – возвращает строку с информацией о книге.

Часть 2: Инкапсуляция и свойства

Инкапсуляция скрывает внутреннюю реализацию и предоставляет контролируемый доступ к атрибутам.

```
class BankAccount:
```

```
    def __init__(self, owner, balance=0):
```

```
        self.owner = owner
```

```
        self.__balance = balance # приватный атрибут (два подчёркивания)
```

```
# Методы для работы с балансом
```

```
    def deposit(self, amount):
```

```
        if amount > 0:
```

```
            self.__balance += amount
```

```
            self.__log_transaction(f"Пополнение: +{amount}")
```

```
        else:
```

```
            print("Сумма должна быть положительной")
```

```
    def withdraw(self, amount):
```

```
        if 0 < amount <= self.__balance:
```

```
            self.__balance -= amount
```

```
            self.__log_transaction(f"Снятие: -{amount}")
```

```
        else:
```

```
            print("Недостаточно средств или неверная сумма")
```

```
    def __log_transaction(self, message):
```

```

        """Приватный метод для логирования"""

        print(f"Лог: {message}. Баланс: {self.__balance}")

    # Геттер для баланса (только чтение)
    def get_balance(self):
        return self.__balance

# Использование
acc = BankAccount("Иван", 1000)
acc.deposit(500)
acc.withdraw(200)
print(acc.get_balance()) # 1300
# print(acc.__balance) # Ошибка! AttributeError

В для управления доступом часто используют декоратор @property.

class Temperature:
    def __init__(self, celsius):
        self._celsius = celsius # защищённый атрибут (одно подчёркивание –
соглашение)

    @property
    def celsius(self):
        return self._celsius

    @celsius.setter
    def celsius(self, value):
        if value < -273.15:
            raise ValueError("Температура ниже абсолютного нуля невозможна")
        self._celsius = value

```

```
@property
```

```
def fahrenheit(self):
```

```
    return self._celsius * 9/5 + 32
```

```
t = Temperature(25)
```

```
print(t.celsius)      # 25
```

```
print(t.fahrenheit)   # 77.0
```

```
t.celsius = 30
```

```
print(t.fahrenheit)   # 86.0
```

```
# t.celsius = -300     # ValueError
```

Задание 2.1: Реализуйте класс Student с приватными атрибутами `__name`, `__grades` (список оценок). Создайте методы:

- `add_grade(grade)` – добавляет оценку (от 2 до 5).
- `average_grade()` – возвращает средний балл.
- Используйте `@property` для получения имени и списка оценок (только для чтения, копию списка).

Часть 3: Наследование

Наследование позволяет создавать новый класс на основе существующего.

```
class Animal:
```

```
    def __init__(self, name):
```

```
        self.name = name
```

```
    def speak(self):
```

```
        raise NotImplementedError("Подкласс должен реализовать этот метод")
```

```
class Cat(Animal):
```

```
    def speak(self):
```

```
return f"{self.name} говорит Мяу!"
```

```
class Dog(Animal):
```

```
    def speak(self):
```

```
        return f"{self.name} говорит Гав!"
```

```
animals = [Cat("Барсик"), Dog("Рекс")]
```

```
for animal in animals:
```

```
    print(animal.speak())
```

Расширение функциональности с помощью super()

```
class Vehicle:
```

```
    def __init__(self, brand, model):
```

```
        self.brand = brand
```

```
        self.model = model
```

```
    def info(self):
```

```
        return f"{self.brand} {self.model}"
```

```
class Car(Vehicle):
```

```
    def __init__(self, brand, model, doors):
```

```
        super().__init__(brand, model) # вызов конструктора родителя
```

```
        self.doors = doors
```

```
    def info(self):
```

```
        # Переопределение метода
```

```
        return f"{super().info()}, дверей: {self.doors}"
```

```
car = Car("Toyota", "Camry", 4)
```

```
print(car.info()) # Toyota Camry, дверей: 4
```

Задание 3.1: Создайте базовый класс Shape с методом area(), который возвращает 0. Реализуйте классы Rectangle (прямоугольник) и Circle, переопределяющие метод area(). Добавьте класс Square, наследующий от Rectangle.

Часть 4: Полиморфизм и магические методы

Полиморфизм позволяет объектам разных классов обрабатывать данные единообразно. Магические методы (**методы**) определяют поведение объектов для встроженных операций.

```
class Vector:
```

```
    def __init__(self, x, y):
```

```
        self.x = x
```

```
        self.y = y
```

```
    def __repr__(self):
```

```
        """Официальное строковое представление (для разработчиков)"""
```

```
        return f"Vector({self.x}, {self.y})"
```

```
    def __str__(self):
```

```
        """Неформальное строковое представление (для пользователей)"""
```

```
        return f"({self.x}, {self.y})"
```

```
    def __add__(self, other):
```

```
        """Перегрузка оператора +"""
```

```
        return Vector(self.x + other.x, self.y + other.y)
```

```
    def __sub__(self, other):
```

```
        return Vector(self.x - other.x, self.y - other.y)
```

```

def __mul__(self, scalar):
    """Умножение на число"""
    return Vector(self.x * scalar, self.y * scalar)

def __eq__(self, other):
    return self.x == other.x and self.y == other.y

def __len__(self):
    """Возвращает длину вектора (норму) – округлённо до целого"""
    return int((self.x ** 2 + self.y ** 2) ** 0.5)

v1 = Vector(2, 3)
v2 = Vector(1, 1)
print(v1)      # (2, 3)
print(v1 + v2)  # (3, 4)
print(v1 * 3)   # (6, 9)
print(len(v1))  # 3 (так как  $\sqrt{13} \approx 3.6 \rightarrow \text{int } 3$ )

```

Другие полезные магические методы:

- `__getitem__(self, key)` – доступ по индексу `obj[key]`
- `__setitem__(self, key, value)` – установка значения по индексу
- `__iter__(self)` – возвращает итератор
- `__call__(self, ...)` – позволяет вызывать объект как функцию

Задание 4.1: Реализуйте класс `Matrix`, который представляет матрицу 2x2. Перегрузите операторы `+`, `-`, `*` (умножение на другую матрицу), `==`. Добавьте метод `__repr__` для красивого вывода.

Часть 5: Классовые и статические методы

- **Метод класса** (`@classmethod`) – работает с классом, а не с экземпляром. Получает класс как первый аргумент (`cls`).

- **Статический метод** (@staticmethod) – обычная функция внутри класса, не получает ни self, ни cls.

```
class Date:
```

```
    def __init__(self, day, month, year):
```

```
        self.day = day
```

```
        self.month = month
```

```
        self.year = year
```

```
    @classmethod
```

```
    def from_string(cls, date_str):
```

```
        """Альтернативный конструктор: создаёт объект из строки 'дд-мм-ггг'"""
```

```
        day, month, year = map(int, date_str.split('-'))
```

```
        return cls(day, month, year)
```

```
    @staticmethod
```

```
    def is_valid_date(date_str):
```

```
        """Проверяет, является ли строка корректной датой (без создания объекта)"""
```

```
        try:
```

```
            day, month, year = map(int, date_str.split('-'))
```

```
            return 1 <= day <= 31 and 1 <= month <= 12 and year > 0
```

```
        except:
```

```
            return False
```

```
d1 = Date.from_string("15-08-2023")
```

```
print(d1.day, d1.month, d1.year) # 15 8 2023
```

```
print(Date.is_valid_date("32-01-2020")) # False
```

Задание 5.1: Добавьте в класс Book из задания 1.1 класс-метод from_dict(data), создающий книгу из словаря, и статический метод is_valid_year(year), проверяющий, что год не больше текущего.

Часть 6: Композиция и агрегация

Объекты могут содержать другие объекты. Это называется композицией (жёсткая связь) или агрегацией (слабая связь).

```
class Author:
```

```
    def __init__(self, name, nationality):
```

```
        self.name = name
```

```
        self.nationality = nationality
```

```
class Book:
```

```
    def __init__(self, title, author, year):
```

```
        self.title = title
```

```
        self.author = author # агрегация: автор создаётся отдельно и может существовать без книги
```

```
        self.year = year
```

```
    def get_info(self):
```

```
        return f'"{self.title}" - {self.author.name} ({self.author.nationality}), {self.year}'
```

```
author = Author("Лев Толстой", "русский")
```

```
book = Book("Война и мир", author, 1869)
```

```
print(book.get_info())
```

Задание 6.1: Создайте класс Library, который содержит список книг. Реализуйте методы:

- `add_book(book)` – добавить книгу.
- `remove_book(title)` – удалить по названию.
- `find_by_author(author_name)` – вернуть список книг указанного автора.
- `borrow_book(title)` – отметить книгу как недоступную.
- `return_book(title)` – вернуть книгу.

